

EXPERIMENT – 5

Name: Sam Fawaz Hadi AL-Basara

UID: 24BAI70236

Section/Group: 24AIT_NTPP 2 (A)

Branch/Semester: CSE_AIML (CS221) / 5th

Subject Code: 24 CSP-337

Subject Name: Full Stack - II

Date of Performance: 21/8/2026

Submitted to: Ms. Lovleen Kaur (E20454)

Experiment – 5.1

1. **AIM :** To design and implement RESTful APIs using Spring Boot with proper validation, standardized responses, and scalable architecture.

2. OBJECTIVE

- 1- To understand REST API design principles
- 2- To implement CRUD APIs using Spring Boot
- 3- To enforce consistent request-response structures
- 4- To apply validation using Bean Validation
- 5- To enable secure cross-origin communication

3. SOFTWARE REQUIREMENT

- 1- Java (JDK 17+)
- 2- Spring Boot
- 3- IDE (IntelliJ / Eclipse / VSCode)
- 4- Postman / API testing tool

4. THEORY:

REST (Representational State Transfer) is an architectural style used for designing scalable and loosely coupled web services. In RESTful systems, data is treated as resources and accessed through standard HTTP methods such as GET, POST, PUT, and DELETE.

Spring Boot simplifies REST API development by providing a layered architecture consisting of:

- **Controller Layer** → Handles HTTP requests
- **Service Layer** → Contains business logic
- **Repository Layer** → Interacts with the database

A well-designed API must follow:

- **Stateless communication**
- **Consistent URI structure**
- **Standardized response format**

To ensure data integrity, **Bean Validation** is applied using annotations such as `@NotNull`, `@NotBlank`, and `@Size`, which validate incoming request data before processing.

Additionally, APIs must support **Cross-Origin Resource Sharing (CORS)** to allow secure communication between frontend and backend systems running on different origins.

Standardizing API responses ensures consistency and improves frontend integration by maintaining a predictable structure for success and error responses.

5. IMPLEMENTATION/PROCEDURE:

1. **Install the required software:** JDK 25, Homebrew, Maven, Visual Studio Code, Google Chrome, and Postman.
2. **Create a Spring Boot project** using Spring Initializr with Maven, Java, Spring Boot, and the required dependencies: Spring Web, Spring Data JPA, H2 Database, and Validation.
3. **Open the generated project** in VS Code and create the required layered package structure: config, controller, dto, exception, model, repository, and service.
4. **Create the User entity** in the Model layer and define uid and name fields with JPA and validation annotations.
5. **Create the Repository layer** using JpaRepository to provide database CRUD operations.
6. **Create DTO classes** (UserRequest and ApiResponse) to validate incoming data and maintain a standardized API response format.
7. **Implement the Service layer** to handle business logic for creating, retrieving, updating, and deleting users.
8. **Implement the Controller layer** and map REST endpoints using POST, GET, PUT, and DELETE HTTP methods.
9. **Implement global exception handling** to return standardized error responses for validation failures and missing users.
10. **Configure CORS** so that the frontend can communicate with the Spring Boot backend from a different origin.
11. **Configure the H2 database** in application.properties and enable the H2 console for database inspection.
12. **Run the Spring Boot application** using `./mvnw spring-boot:run` and verify that the REST API is available at localhost:8080.
13. **Test the REST API using Postman** by performing Create, Read, Update, and Delete operations and checking the returned responses.
14. **Develop the frontend** using HTML, CSS, and JavaScript with a form for adding users and a table for displaying user records.
15. **Connect the frontend to the REST API** using JavaScript `fetch()` requests and implement Add, Refresh, Edit, Update, and Delete operations.
16. **Run the frontend using Live Server** and open it in Google Chrome.
17. **Test the complete application** by verifying CRUD operations, validation, standardized responses, CORS communication, and H2 database storage.

18 · **Confirm the expected outcome:** a functional Spring Boot RESTful User Management System with a layered architecture, validation, standardized API responses, database integration, and frontend-backend communication.

5. CODE

User.java

```
@Entity
@Table(name = "users")
public class User {

    @Id
    private String uid;

    @NotBlank
    @Size(min = 2, max = 50)
    private String name;

    public User() {}

    public User(String uid, String name) {
        this.uid = uid;
        this.name = name;
    }

    // Getters and Setters
}
```

UserRepository.java

```
public interface UserRepository
    extends JpaRepository<User, String> {
}
```

UserService.java

```
@Service

public class UserService {
```

```
private final UserRepository repository;
```

```
public UserService(UserRepository repository) {  
    this.repository = repository;  
}
```

```
public User create(User user) {  
    return repository.save(user);  
}
```

```
public List<User> getAll() {  
    return repository.findAll();  
}
```

```
public User getById(String uid) {  
    return repository.findById(uid)  
        .orElseThrow(() ->  
            new RuntimeException("User not found"));  
}
```

```
public User update(String uid, User user) {  
    User existing = getById(uid);  
    existing.setName(user.getName());  
    return repository.save(existing);  
}
```

```
public void delete(String uid) {  
  
    repository.delete(getById(uid));  
  
}  
  
}
```

UserController.java

```
@RestController  
@RequestMapping("/api/users")  
@CrossOrigin(origins = "*")  
public class UserController {  
  
    private final UserService service;  
  
    public UserController(UserService service) {  
        this.service = service;  
    }  
  
    @PostMapping  
    public User create(@RequestBody User user) {  
        return service.create(user);  
    }  
  
    @GetMapping  
    public List<User> getAll() {  
        return service.getAll();  
    }  
  
    @GetMapping("/{uid}")  
    public User get(@PathVariable String uid) {  
        return service.getById(uid);  
    }  
  
    @PutMapping("/{uid}")  
    public User update(
```

```
    @PathVariable String uid,
    @RequestBody User user) {
    return service.update(uid, user);
}

@DeleteMapping("/{uid}")
public String delete(@PathVariable String uid) {
    service.delete(uid);
    return "User deleted successfully";
}
}
```

Frontend Main Part – script.js

```
const API_URL = "http://localhost:8080/api/users";
```

```
// CREATE
```

```
fetch(API_URL, {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({
    uid: "101",
    name: "Sam"
  })
});
```

```
// READ
```

```
fetch(API_URL)
  .then(response => response.json())
  .then(data => console.log(data));
```

```
// UPDATE
```

```
fetch(`${API_URL}/101`, {
  method: "PUT",
```

```
headers: {"Content-Type": "application/json"},
body: JSON.stringify({
  uid: "101",
  name: "Updated Name"
})
});

// DELETE
fetch(`${API_URL}/101`, {
  method: "DELETE"
});
```

6. OUTPUT

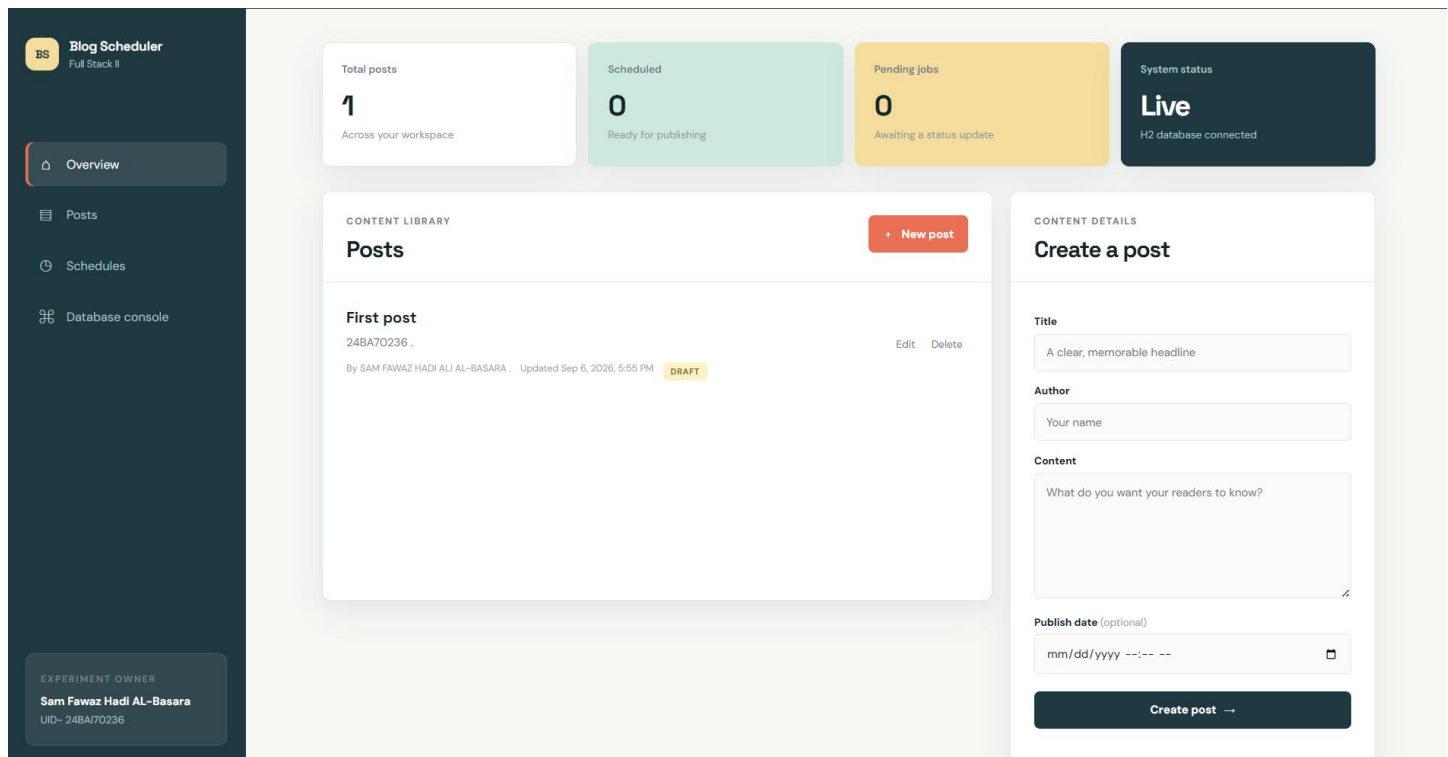


Fig 1-Final REST API User Management System – Spring Boot Backend with Frontend Interface

7. LEARNING OUTCOME

- Understand the basic principles of **RESTful API architecture**.
- Develop **CRUD REST APIs** using Spring Boot.
- Implement **layered architecture** using Controller, Service, and Repository layers.
- Apply **Bean Validation** to validate API requests.
- Implement **standardized API responses and exception handling**.
- Configure **CORS and H2 database** for frontend-backend integration.
- Test REST APIs using **Postman and a web frontend**.

Experiment – 5.2

1. **AIM:** To implement global exception handling and structured logging for building robust and observable backend systems.

2. OBJECTIVE

- 1- To handle exceptions centrally using `@ControllerAdvice`
- 2- To implement logging mechanisms for request tracking
- 3- To use correlation IDs for tracing requests
- 4- To improve system observability and debugging

3. SOFTWARE REQUIREMENT

- 1- Java (JDK 17+)
- 2- Spring Boot
- 3- IDE (IntelliJ / Eclipse / VSCode)
- 4- Postman / API testing tool
- 5- Logging framework (SLF4J / Logback)

4. THEORY

In large-scale applications, handling exceptions individually in each controller leads to code duplication and inconsistency. To address this, Spring Boot provides **global exception handling** using `@ControllerAdvice`, which centralizes error handling and ensures uniform error responses across the application.

This approach improves:

- Code maintainability

- Error consistency
- Debugging efficiency

In addition to exception handling, **logging** plays a crucial role in monitoring and debugging backend systems. Logging mechanisms capture details such as request paths, execution time, and error information.

Filters and Interceptors act as middleware components that intercept incoming requests and outgoing responses. They are commonly used for:

- Logging
- Authentication
- Request modification

To enhance traceability, a **correlation ID** is assigned to each request using MDC (Mapped Diagnostic Context). This allows developers to track the complete lifecycle of a request across logs, which is especially important in distributed systems.

Together, global exception handling and structured logging contribute to building a **robust, observable, and production-ready backend system**.

5. IMPLEMENTATION/PROCEDURE

1. Continue using the **Experiment 5.1 Spring Boot project**.
2. Create a `UserNotFoundException` class extending `RuntimeException`.
3. Modify `UserService` to throw the custom exception when a UID is not found.
4. Create `GlobalExceptionHandler` using `@RestControllerAdvice`.
5. Handle validation errors, user-not-found errors, and general exceptions centrally.
6. Create a `LoggingFilter` using `OncePerRequestFilter`.
7. Generate a unique correlation ID for every HTTP request.
8. Store the correlation ID using MDC.
9. Log request method, URI, response status, and execution time.
10. Configure the console logging pattern in `application.properties`.
11. Update the frontend to support **Add, Search, Refresh, Edit, Update, Delete, and Clear** operations.
12. Start the Spring Boot application.
13. Test valid and invalid user requests.
14. Verify the API response and correlation ID.
15. Observe the request logs in the Spring Boot terminal.

6. CODE

`UserNotFoundException.java`

```
package com.example.experiment5.exception;
```

```
public class UserNotFoundException extends RuntimeException {
```

```
    public UserNotFoundException(String message) {  
        super(message);  
    }  
}
```

UserService.java

```
public User getUserById(String uid) {
```

```
    return userRepository.findById(uid)  
        .orElseThrow(() ->  
            new UserNotFoundException(  
                "User not found with UID: " + uid  
            )  
        );  
}
```

```
public User updateUser(String uid, UserRequest request) {
```

```
    User user = getUserById(uid);  
    user.setName(request.getName());  
  
    return userRepository.save(user);  
}
```

```
public void deleteUser(String uid) {
```

```
    User user = getUserById(uid);  
    userRepository.delete(user);  
}
```

GlobalExceptionHandler.java

@RestControllerAdvice

```
public class GlobalExceptionHandler {
```

```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<ApiResponse<String>>
handleUserNotFound(UserNotFoundException exception) {

    return ResponseEntity
        .status(HttpStatus.NOT_FOUND)
        .body(new ApiResponse<>(
            false,
            exception.getMessage(),
            null
        ));
}
```

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ApiResponse<String>>
handleValidation(MethodArgumentNotValidException exception) {
```

```
    String message = exception.getBindingResult()
        .getFieldErrors()
        .get(0)
        .getDefaultMessage();
```

```
    return ResponseEntity
        .badRequest()
        .body(new ApiResponse<>(
            false,
            message,
            null
        ));
}
```

LoggingFilter.java

```
@Component
```

```
public class LoggingFilter extends OncePerRequestFilter {
```

```
    private static final Logger logger =
        LoggerFactory.getLogger(LoggingFilter.class);
```

```
@Override
protected void doFilterInternal(
    HttpServletRequest request,
    HttpServletResponse response,
    FilterChain filterChain)
    throws ServletException, IOException {

    String correlationId =
        UUID.randomUUID().toString();

    long startTime = System.currentTimeMillis();

    MDC.put("correlationId", correlationId);

    logger.info(
        "Request started | method={} | uri={}",
        request.getMethod(),
        request.getRequestURI()
    );

    try {
        filterChain.doFilter(request, response);
    }
    finally {

        long executionTime =
            System.currentTimeMillis() - startTime;

        response.setHeader(
            "X-Correlation-ID",
            correlationId
        );

        logger.info(
            "Request completed | status={} | executionTime={} ms",
            response.getStatus(),
            executionTime
        );
    }
}
```

```
);
```

```
MDC.remove("correlationId");
```

```
}
```

```
}
```

```
}
```

```
application.properties
```

```
server.port=8080
```

```
spring.datasource.url=jdbc:h2:mem:experiment5
```

```
spring.datasource.driver-class-name=org.h2.Driver
```

```
spring.datasource.username=sa
```

```
spring.datasource.password=
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

```
spring.h2.console.enabled=true
```

```
spring.h2.console.path=/h2-console
```

```
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} [correlationId=%X{correlationId}] -  
%msg%n
```

```
logging.level.com.example.experiment5=INFO
```

```
script.js
```

```
const API_URL = "http://localhost:8080/api/users";
```

```
// Load users
```

```
async function loadUsers() {
```

```
    const response = await fetch(API_URL);
```

```
    const result = await response.json();
```

```
    if (result.success) {
```

```
        // Display users in table
```

```
    }
```

```
}
```

```
// Search user
```

```
async function searchUser() {
```

```
    const uid = document.getElementById("searchUid")
```

```
        .value.trim();
```

```
const response =  
  await fetch(`${API_URL}/${uid}`);  
  
const result = await response.json();  
  
if (result.success) {  
  // Display user  
} else {  
  // Display exception message  
  // Example: User not found with UID: INVALID123  
}  
}  
  
// Delete user  
async function deleteUser(uid) {  
  
  const response = await fetch(  
    `${API_URL}/${uid}`,  
    { method: "DELETE" }  
  );  
  
  const result = await response.json();  
  
  if (result.success) {  
    await loadUsers();  
  }  
}  
  
{  
  "success": false,  
  "message": "User not found with UID: INVALID123",  
  "data": null  
}
```

7. OUTPUT

User records, properly traced.

A working Spring Boot lab for global exception handling, structured request logs, CRUD operations, and correlation-based debugging.

SUBMITTED BY: Sam Fawaz Hadi AL-Basara UID: 24BAI70236

1 REGISTERED USERS
1 REQUESTS THIS SESSION
MDC TRACE CONTEXT

01 / USER REGISTRY
NEW RECORD

Create a user

University UID

Full name

Add user +
Clear

UIDs are unique. Select Edit from the table to update a name.

02 / REQUEST CONTEXT
MDC

Correlation ID

Trace label for the next request

lab-session-337 X-CORRELATION-ID

The filter echoes this ID in every response and places it in the structured logging context.

03 / LOOKUP
GET

Find a user

Search ↗

Enter a UID to inspect a single record.

04 / CONTROLLED FAILURE
404

Exception lab

Request a missing user and inspect the standardized `ApiError` envelope from `@RestControllerAdvice`.

Trigger not-found response ↗

05 / LIVE RECORDS
Refresh list

Registered users

UID	NAME	ACTIONS
24BAI70236	Sam Fawaz Hadi AL-Basara	Edit Delete

Fig. 5.2.1 – Final User Management Application with Global Exception Handling and Structured Logging

8. LEARNING OUTCOME

- · Implement centralized exception handling in Spring Boot.
- · Create and use custom runtime exceptions.
- · Develop request-logging filters.
- · Generate and use correlation IDs with MDC.
- · Trace API requests using structured logs.
- · Build robust REST APIs with consistent error responses.
- · Integrate frontend CRUD operations with a Spring Boot backend.

Sam 24BAI70236